

Project Disaster Management & Alert System

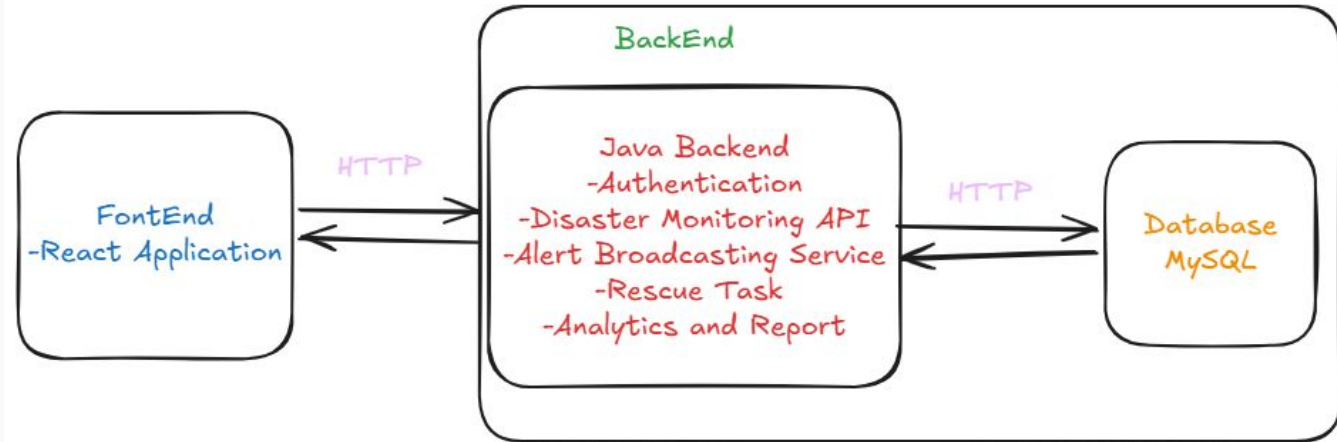
Platform designed to bridge communication gaps between authorities, first responders, and citizens during emergencies



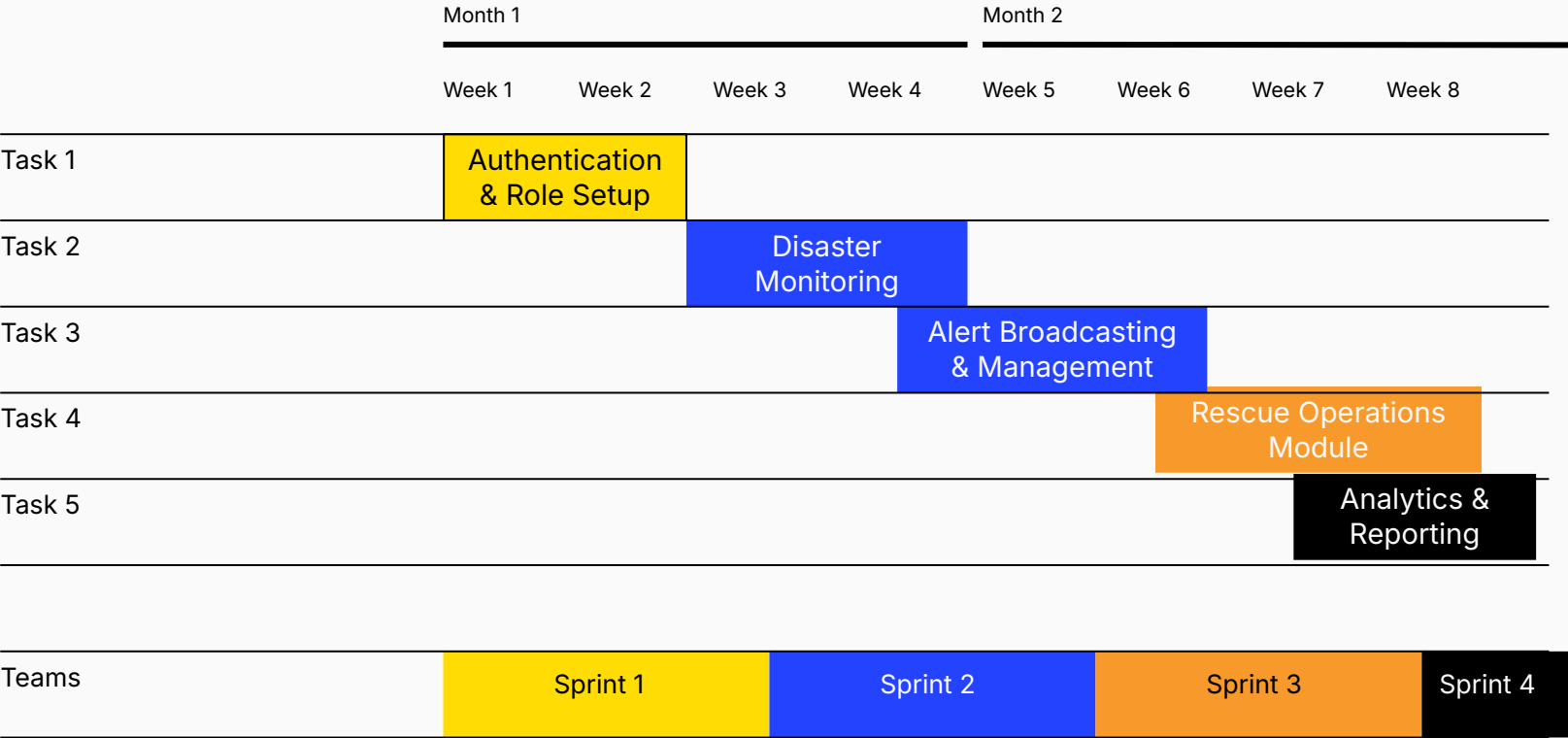
Technical Architecture

1. **Frontend:** React-based web application.
2. **Backend:** Java-based RESTful API service.
3. **Security:** JWT-based (JSON Web Token) authentication for secure access control, Bcrypt for hashing.
4. **Database:** MySQL or PostgreSQL for storing users, disaster logs, and tasks.

System Design Diagram



Project workflow



Project Modules

1

Authentication & Role Management

JWT-based authentication for secure login and access control.

Role-based permissions for Admin, Responder, and Citizen.

Profile setup with personal details, contact info, and location

2

Disaster Monitoring

Integration with external APIs (e.g., OpenWeather, NDMA feeds) for real-time disaster data.

Categorize disasters (flood, earthquake, fire, cyclone, etc.).

3

Rescue Task Assignment

Admin assigns responders to affected zones.

Responders update task progress (Pending, Ongoing, Completed)

Citizens can request rescue via form submission.

4

Analytics & Reporting

Generate reports for disaster types, affected areas, and response times.

Visual dashboards showing alert frequency and responder activity.

Comparative data to evaluate preparedness and performance.

Work in the Milestone 1 and 2

01

Implement JWT authentication and Role-based access
(Admin, Responder, Citizen)

02

Profile setup with details and location

03

Disaster Monitoring Integration with external APIs



Project phase 1 :

Authentication and Role based access

- Secure login and registration system using JSON Web Tokens (JWT).
- Ensure protected endpoints for role-based access and validation.
- Admin controls system configuration and task allocation.
- Responders manage assigned rescue operations.
- Citizens can view alerts and request help.

`http://localhost:5173`

Key	Value
role	ROLE_CITIZEN
token	eyJhbGciOiJIUzI...
userId	2



Project phase 2:

Profile setup with details and location

- Capture user name, phone number, and region.
- Store data in the database for disaster-based notification targeting.
- Send Notification Based on Location of Citizens

DMAS PORTAL

Disaster Management & Alert System

Login

Register

FULL NAME

EMAIL ADDRESS

PASSWORD

PHONE

LOCATION

City, State

Auto

ACCOUNT ROLE

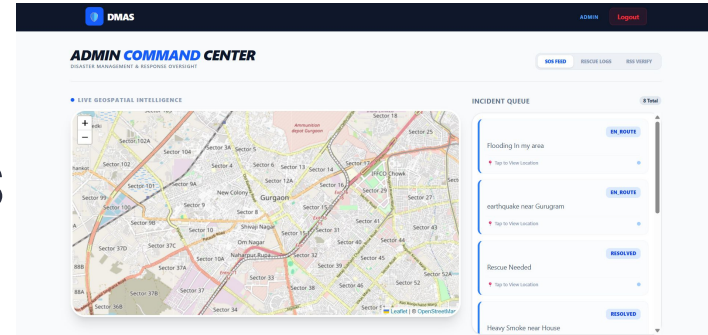
Citizen

CREATE ACCOUNT

Project phase 3:

Disaster Monitoring Integration with external APIs

- Connect to live disaster alert APIs (e.g., weather, flood, earthquake).
- Parse and store real-time event data in the system.
- Automatically tag incoming data as "Flood", "Cyclone", etc.
- Allow Admins to verify and edit alert details before broadcast



Project Modules

1

Authentication & Role Management

JWT-based authentication for secure login and access control.

Role-based permissions for Admin, Responder, and Citizen.

Profile setup with personal details, contact info, and location

2

Disaster Monitoring

Integration with external APIs (e.g., OpenWeather, NDMA feeds) for real-time disaster data.

Categorize disasters (flood, earthquake, fire, cyclone, etc.).

3

Rescue Task Assignment

Admin assigns responders to affected zones.

Responders update task progress (Pending, Ongoing, Completed)

Citizens can request rescue via form submission.

4

Analytics & Reporting

Generate reports for disaster types, affected areas, and response times.

Visual dashboards showing alert frequency and responder activity.

Comparative data to evaluate preparedness and performance.

Work in the Milestone 3



01

The Citizen's Request for Help



02

The Admin's Command Center



03

The Responder's Action



Project phase 1: Citizen's Request for Help

The Concept: Real-time reporting.

The Demo: Show the Citizen Dashboard. Click the SOS button.

EMERGENCY ACTIONS



Emergency Alert

Click to report a life-threatening situation



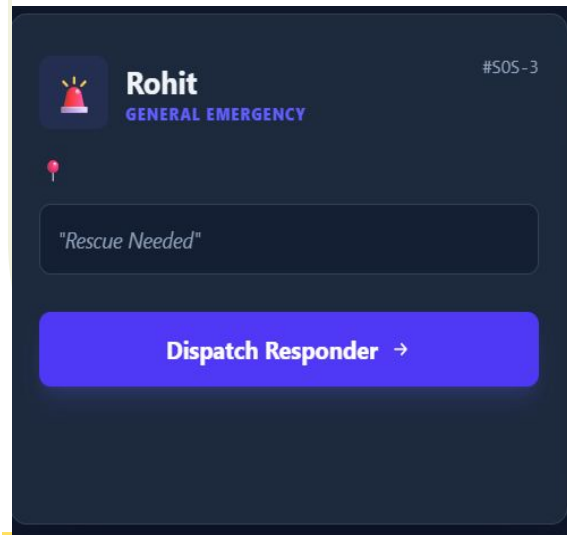
Project phase 2:

The Admin's Command Center

The Concept: Centralized Dispatch.

The Demo: Open the Admin Dashboard (Dark Theme).

the new SOS request appearing at the top of the list.





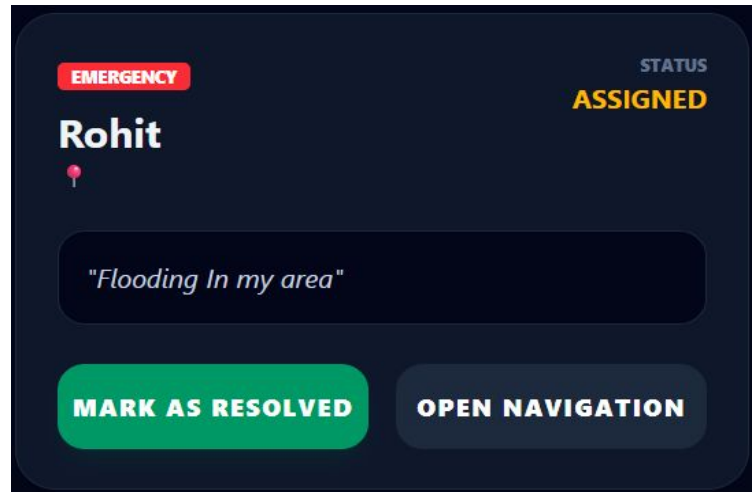
Project phase 3:

The Responder's Action

The Concept: Field Operations.

The Demo: Switch to the Responder view.

Show the assigned task appearing. Click the "Open Navigation" button to show Google Maps integration.



Project Modules

1

Authentication & Role Management

JWT-based authentication for secure login and access control.

Role-based permissions for Admin, Responder, and Citizen.

Profile setup with personal details, contact info, and location

2

Disaster Monitoring

Integration with external APIs (e.g., OpenWeather, NDMA feeds) for real-time disaster data.

Categorize disasters (flood, earthquake, fire, cyclone, etc.).

3

Rescue Task Assignment

Admin assigns responders to affected zones.

Responders update task progress (Pending, Ongoing, Completed)

Citizens can request rescue via form submission.

4

Analytics & Reporting

Generate reports for disaster types, affected areas, and response times.

Visual dashboards showing alert frequency and responder activity.

Comparative data to evaluate preparedness and performance.

Work in the Milestone 4

01

Task Assignment & Real-Time Orchestration

02

Geo-Location & Geospatial Intelligence

03

Incident Reporting & Audit Trail

Project phase 1:

Task Assignment & Real-Time Orchestration

- **The Problem:** In disasters, communication is chaotic. Responders often don't know where to go first.
- **Our Solution: Admin Command:** The Admin uses a centralized dashboard to assign SOS requests to specific responders.
- **Responder Dashboard:** A mobile-responsive interface for field staff to accept tasks, and update status (e.g., "En Route," "In Progress," "Resolved").

POST UPDATE

CURRENT STATUS

EN ROUTE

AT LOCATION

CRITICAL

COMPLETED

UPDATE NOTE

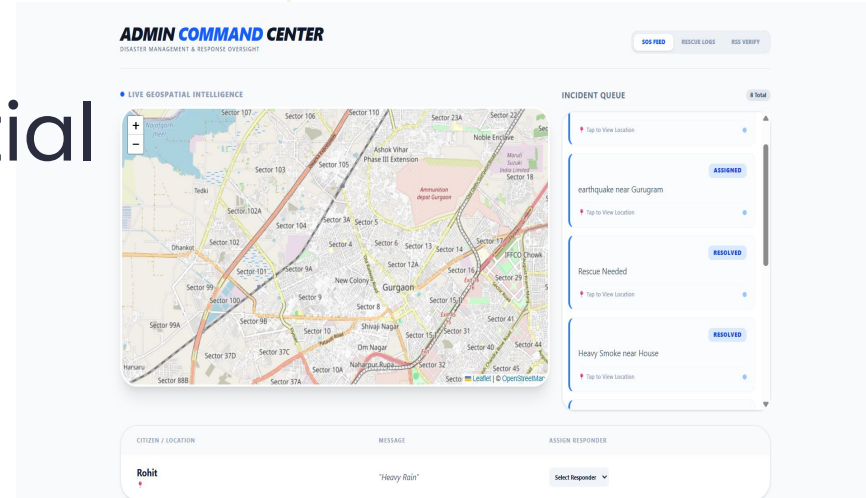
Describe the current situation...

CONFIRM & SYNC WITH COMMAND

Project phase 2:

Geo-Location & Geospatial Intelligence

- **Visualization:** We integrated **Leaflet.js** and **OpenStreetMap** to move away from text-only data.
- **"Fly-To" Capability:** Admin selects a rescue task, the map uses a cinematic **Fly-To animation** to zoom directly into the disaster coordinates (\$Latitude, Longitude\$).
- **Spatial Data:** We store precise coordinates of both **Disaster Zones** (RSS Feed data) and the **SOS Sites** (Citizen requests) to create a comprehensive map of the emergency.

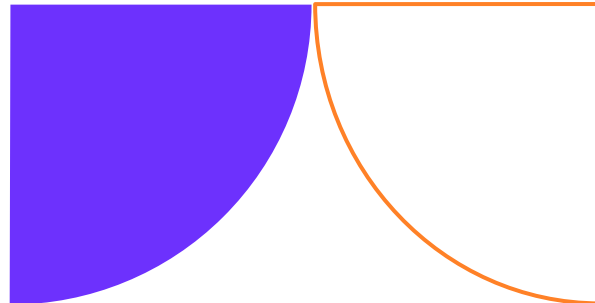
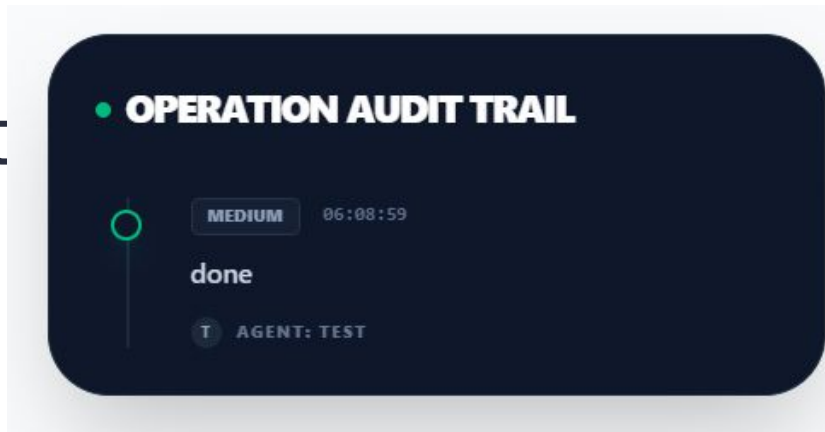




Project phase 3:

Incident Reporting & Audit Trail

- **Transparency:** Every action taken by a responder is **Timestamped** and stored in the database.
- **The Timeline Component:** We developed a vertical timeline view that acts as an **Audit Log**. This shows:
 1. When the SOS was raised.
 2. When the Admin assigned the responder.
 3. Every progress update submitted from the field.
- **Accountability:** This ensures that post-disaster reviews have a clear record of response times and resource allocation.



THE TEAM



<https://github.com/MeRohit0/disaster-management-and-alert-system-infosys-internship>

Thank You 🙌